Somaiya Vidyavihar University

K J Somaiya School of Engineering

Department of Electronics and Telecommunication Engineering

Programme: Computer and Communication Engineering

LY B.Tech – CCE – July to November 2026

DevOps

Internal Assessment–II

SecureMonitor

DevSecOps Security Scanning with Application Monitoring

Student Name	_____
Roll Number	_____

Project repository: github.com/Aditi141004/SecureMonitor

1. Brief Description

SecureMonitor is a small Flask-based application that combines DevSecOps checks with operational monitoring. GitHub Actions runs automated tests and a dependency vulnerability audit as part of the CI workflow. CodeQL provides code-scanning workflow analysis, while OWASP ZAP performs a passive baseline scan of the running web application. Prometheus scrapes application metrics, and Grafana visualizes application health, request rate, errors, response time, and request totals. The project demonstrates how security checks can be integrated before changes proceed through a CI workflow and how the running application can be observed after deployment.

2. Platform and Tools Used

Tool / Platform	Purpose
Python / Flask	Web application and health endpoint
pytest	Automated application tests
Git / GitHub	Version control and repository hosting
GitHub Actions	CI workflow, test job, and dependency security gate
CodeQL	Automated code-scanning workflow
pip-audit	Checks declared Python dependencies for known vulnerabilities
OWASP ZAP	Passive baseline scan of the local web application
Prometheus	Scrapes and queries application metrics
Grafana	Monitoring dashboard and visualization
Docker Desktop	Runs Prometheus and ZAP containers

3. Step-by-Step Workflow

1. Build the application: A Flask application exposes application routes, a health endpoint, and a Prometheus-compatible `/metrics` endpoint.

2. Add automated tests: pytest checks the home page, health endpoint, API data route, API error route, and exported metrics.

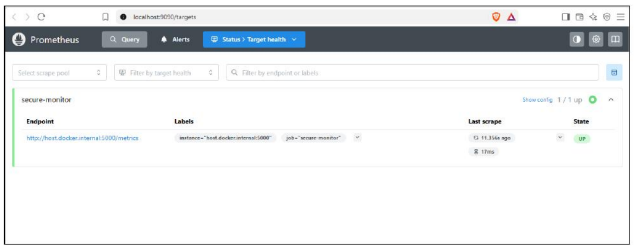
3. Integrate CI: GitHub Actions installs the requirements and runs the automated test suite on pushes and pull requests.

4. Add security checks: CodeQL workflow analysis and OWASP ZAP baseline scanning are used alongside `pip-audit` dependency checks.

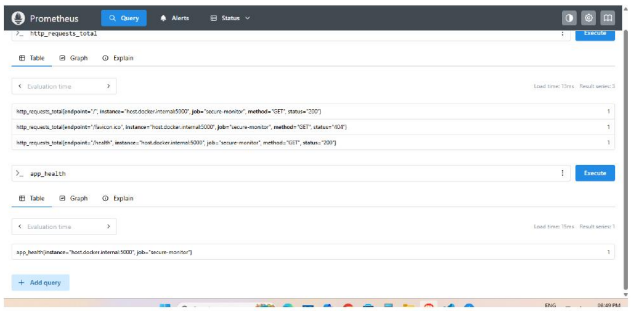
5. Enforce the security gate: The `security-gate` job depends on the test job and runs a strict dependency audit. An audit failure causes the job and workflow to fail.

6. Monitor the application: Prometheus scrapes metrics from Flask; Grafana queries Prometheus and displays health, request totals, request rate, error rate, and response time.

4. Screenshots and Results — Prometheus



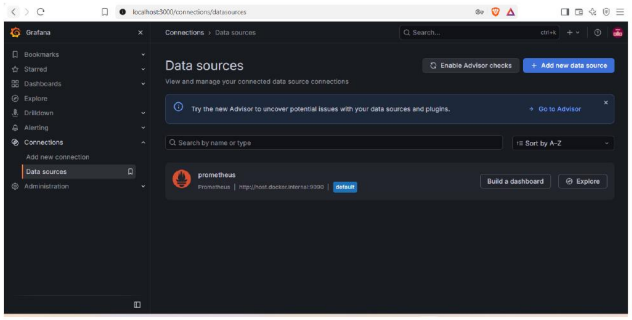
Prometheus Target Health Verification — The Prometheus dashboard confirms that the `secure-monitor` target is successfully connected to the Flask application. The `/metrics` endpoint is being scraped successfully, and the target status is **UP (1/1)**, confirming successful integration between Flask and Prometheus.



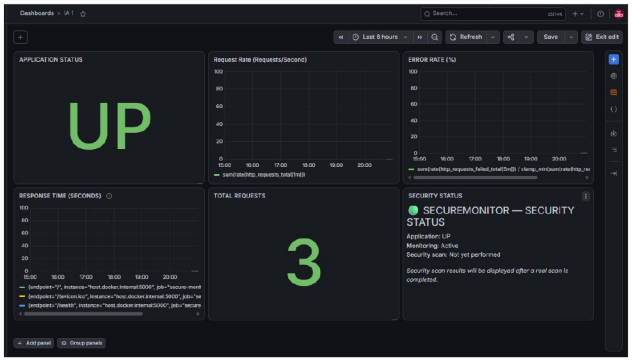
Prometheus Metrics Query Verification — The Prometheus query interface successfully displays the `http_requests_total` and `app_health` metrics collected from the Flask application. The `http_requests_total` metric shows three request series for the `/`, `/favicon.ico`, and `/health` endpoints, while `app_health` returns a value of `1`, indicating that the application is healthy.

Figure 1. Prometheus target and metric verification. The secure-monitor target is shown as UP, and the Prometheus query interface displays application metrics including request counts and health.

5. Screenshots and Results — Grafana



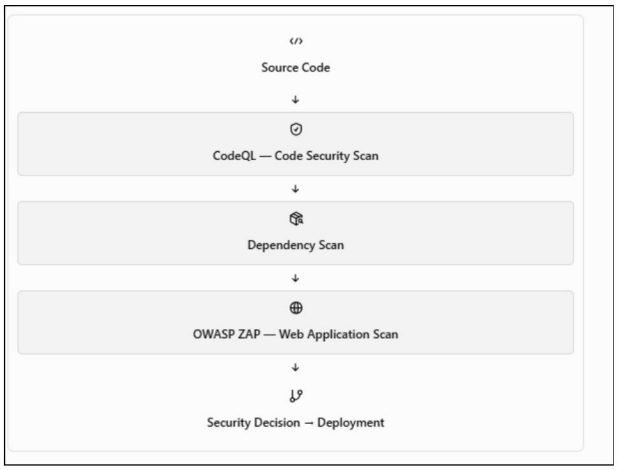
Prometheus added as a data source on Grafana



SecureMonitor Grafana Monitoring Dashboard — The Grafana dashboard integrates with Prometheus to visualize Flask application health, request rate, error rate, response time, and total request count. The Security Status panel summarizes the current monitoring state and indicates that a security scan has not yet been performed. The dashboard provides a centralized view of application performance and operational health.

Figure 2. SecureMonitor Grafana dashboard and Prometheus data-source configuration. The dashboard summarizes application status and request metrics in one view.

6. DevSecOps Workflow and ZAP Scan



DevSecOps pipeline

Figure 3. Security workflow overview showing source code, security checks, dependency scanning, a security decision, and deployment.

OWASP ZAP Baseline Scan

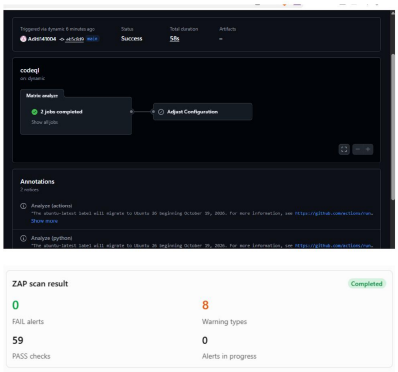
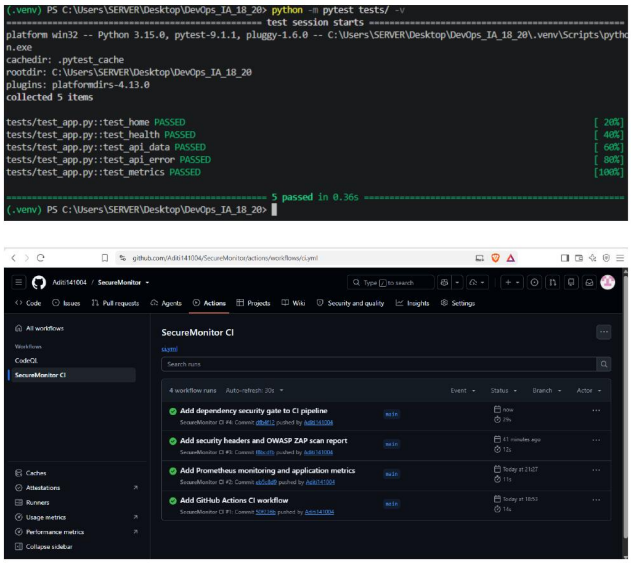


Figure 4. OWASP ZAP baseline scan evidence. The second scan reported 62 PASS checks, five warning types, and zero FAIL alerts. The warnings still require review; zero FAIL alerts does not establish that the application is vulnerability-free.

7. Automated Tests and CI Workflow

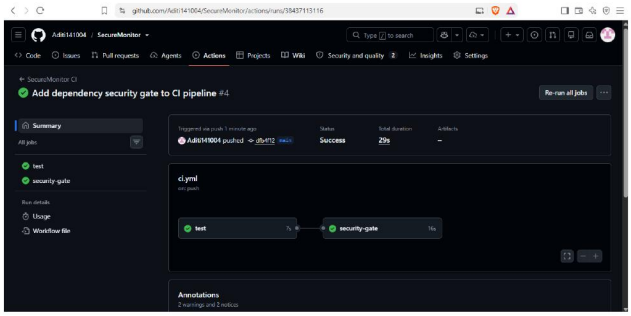


SecureMonitor CI Pipeline Execution on GitHub Actions

The GitHub Actions dashboard displays successful execution of the SecureMonitor CI workflow following the addition of the dependency security gate. The latest workflow run, associated with commit `dfb4f12`, completed successfully. Previous runs for the CI workflow, Prometheus monitoring integration, and OWASP ZAP security report also show successful completion.

Figure 5. GitHub Actions CI history and terminal test evidence. The five pytest tests passed, and the CI workflow runs the test job on repository changes.

8. Security Gate Verification



Successful Execution of the SecureMonitor DevSecOps Security Gate

The GitHub Actions workflow completed successfully for commit `dfb4f12`. Both the automated testing job (`test`) and the dependency security audit job (`security-gate`) passed. The security gate checks Python dependencies for known vulnerabilities using `pip-audit` and prevents the workflow from succeeding if the audit fails.

Figure 6. Successful GitHub Actions run for commit dfb4f12. Both the test job and security-gate job completed successfully; the gate uses pip-audit to check dependencies for known vulnerabilities.

9. Results

Verification	Observed result
Automated tests	5 passed
Local dependency audit	No known vulnerabilities found by pip-audit at the time of the check
OWASP ZAP baseline scan	62 PASS checks, 5 warning types, 0 FAIL alerts on the second scan
GitHub Actions	Both test and security-gate jobs passed in the recorded run
Prometheus	Application target verified UP and metrics queried successfully
Grafana	Dashboard displays application status and request/performance metrics

10. Conclusion / Takeaway

SecureMonitor demonstrates a practical integration of DevOps automation, security checks, and application observability. Automated tests and a dependency audit run in GitHub Actions, while CodeQL and OWASP ZAP contribute additional security analysis. Prometheus and Grafana provide visibility into the application's runtime behavior. The completed checks passed, but the ZAP warnings show that further hardening and review remain appropriate. The main learning is that security validation should be part of the development workflow, while monitoring helps assess the health of the running service.

Repository: https://github.com/Aditi141004/SecureMonitor